A design problem that occurred during the development of the databaseManager is that many parts of the system read and write to the same local database which include ImportService, BudgetService, GoalService, CategorizationService, SpendingTool, SubscriptionService, and ReportService. If each created its own connection manager, the app would open redundant connections. In order to prevent this issue, the system uses the singleton pattern to ensure that only one DatabaseManager instance exists. As a result, all seven classes share one database connection and always read and write consistent data. This design pattern is appropriate because it enforces the constraint of having exactly one connection manager for the application's single local database. If this pattern is not introduced to the design, the databaseManager would have to be created at runtime and passed through every constructor. The classes involved are ImportService, BudgetService, GoalService, CategorizationService, SpendingTool, SubscriptionService, ReportService and DatabaseManager.

- The role of ImportServices is to get the shared instance and use it to save imported transactions and to look up existing ones when checking for duplicates.
- The role of BudgetService is to use the shared DatabaseManager instance to save and retrieve monthly budgets and to read spending totals when checking whether a category has crossed its warning threshold.
- The role of GoalService is to create, update and retrieve savings goals and the contributions made toward them.
- The role of CategorizationService is to update each transaction's category and to save and load the category rules that the user creates.
- The role of SpendingTool is to query the user's transactions.
- The role of SubscriptionService is to load the transaction history that it scans for repeating charges, and to save the detected subscriptions along with the user’s confirm, rename and “not a subscription” choices.
- The role of ReportService is to load the selected month’s transactions and to save the finished monthly report.
- The role of DatabaseManager is the Singleton itself. Its private constructor prevents other classes from creating new instances.

When the user imports a CSV, ImportService needs a parser that understands that bank's CSV layout. In order to address this requirement, the design uses the factory pattern to let ParserFactory choose which parser to create at runtime. ImportService passes the bank name to ParserFactory, which returns a TDParser or a CIBCParser depending on the bank selected. Without the factory pattern, ImportService would need to contain conditional logic to determine which parser to create. This would make ImportService tightly coupled to every specific parser and would require modifying ImportService when a new bank is added. The classes involved are ImportService, ParserFactory, StatementParser, CSVParser, TDParser, CIBCParser, and ImportResult.

- The role of ParserFactory is the factory. Its createParser method checks the bank name and returns the matching parser object.
- The role of StatementParser is the product interface
- The role of CSVParser is the abstract base class for the products. It implements StatementParser and provides the reading logic that all CSV parsers share
- The roles of TDParser and CIBCParser are the concrete products. These are the objects the factory actually creates

The observer pattern is used in the BudgetService class diagram to notify the user when spending in a category reaches 80% of its budget or goes over it. When checkThresholds(month) finds a category that has crossed the warning threshold, BudgetService calls notifyObservers(message), which sends the alert to every attached AlertObserver. GuiAlertPanel then shows the alert in the dashboard, and CliAlertPrinter prints it to the console. This pattern is appropriate because it allows the subject to notify receivers through a common interface without depending on their concrete classes. Without using this pattern, it will introduce tight coupling because BudgetService would have to call GuiAlertPanel and CliAlertPrinter directly. The classes involved are BudgetService, AlertObserver(interface), GuiAlertPanel, and CliAlertPrinter.

- The role of AlertObserver is the Observer interface. It declares a single method onAlert(message: String) in which every observer must implement.
- The role of BudgetService is the subject and it holds the observer list with attach, detach, and notifyObservers
- GuiAlertPanel implements AlertObserver, and when onAlert() is called, it stores the message in its messages list and displays the alert in the GUI dashboard.
- CliAlertPrinter implements AlertObserver, and when onAlert() is called, it prints the alert message to the console

Several parts of the system need Claude to perform their functions such as the chat agent, categorization, subscription labeling, report summaries, and goal tips. The Anthropic API has its own interface. It takes a request payload (createMessage(payload: Map): Map) and returns a structured reply. The classes only want to send a prompt and get text back. To bridge this gap, the system uses the Adapter pattern. It defines a simple interface, LLMClient, with one method, generate(prompt: String): String. ClaudeClient implements that interface and translates each call into the Anthropic API format. This pattern is appropriate because the rest of the system can stay unaware of the API's request and response format. Without it, FinanceAgent, CategorizationService, SubscriptionService, ReportService and InProgressState would each build Anthropic payloads and parse replies themselves. That would duplicate the same code in five places, and any change to the API, the model, or the provider would mean editing all of them. With the adapter, only ClaudeClient changes. The participating classes are FinanceAgent, LLMClient, ClaudeClient, and AnthropicApi.

- FinanceAgent is the Client. It asks Claude what to do next through LLMClient.generate(). The other classes that call LLMClient (CategorizationService, SubscriptionService, ReportService and InProgressState) are also Clients.
- LLMClient is the Target. It is the simple interface the system expects: send a prompt and get the reply back as text.
- ClaudeClient is the Adapter. It implements LLMClient and holds an AnthropicApi reference, the model name and the API key. Its generate() method converts the prompt into Anthropic's format, calls the API, and converts the reply back into a String.
- AnthropicApi is the Adaptee. It is Anthropic's external service, which has an incompatible interface and is not our own code.

The state pattern is used in the goals design to address the problem of a savings goal needing to behave differently depending on whether it is still in progress or already completed. When a goal is in progress, operations such as contributions are added toward the target, the required monthly contribution is shown, and an AI-generated progress message is displayed. Once the target is reached, the goal should instead show a completion message and stop showing a required contribution. Without the state pattern, the goal would need conditional logic to check in every method to decide which behaviour to use. The classes involved are Goal, GoalState (interface), InProgressState, CompletedState, and LLMClient

- Goal stores the goal's name, target amount, deadline and the amount contributed so far, and it holds a reference to its current state.
- GoalState declares the methods every state must provide
- In progress handles a goal that hasn't reached its target yet and it adds each contribution to the goal's total
- CompletedState handles a goal whose target has been met
- LLMClient is not a part of state but is the interface InProgressState uses to generate its AI message